Leave Analysis Report

Prepared By: Binal Desai

Part A — Insights for Business Owners

1. Top Employees by Leave Type

Measures:	**Total Leave Days Taken** The total number of approved leave days taken by each employee, grouped by leave type (Annual, Sick, Unpaid, Other). TotalDaysTaken SUM(fd.DayCount)
Facts:	FactLeaveDay
Dims:	DimEmployee, DimLeaveType, DimStatus
Explanation:	Identifies the top employees taking the most leave across categories (Annual, Sick, Unpaid, Other). This helps employers notice patterns, like frequent sick or unpaid leave, and identify if someone is taking excessive leave that could affect business operations.

```
SELECT e.EmployeeID, e.FirstName, e.LastName, e.CompanyName,
    lt.LeaveTypeCode,
    SUM(fd.DayCount) AS TotalDaysTaken
FROM FactLeaveDay fd
JOIN DimEmployee e ON fd.EmployeeKey = e.EmployeeKey
JOIN DimLeaveType lt ON fd.LeaveTypeKey = lt.LeaveTypeKey
JOIN DimStatus s ON fd.StatusKey = s.StatusKey
WHERE s.StatusCode = 'APPROVED'
GROUP BY e.EmployeeID, e.FirstName, e.LastName, e.CompanyName, lt.LeaveTypeCode
ORDER BY TotalDaysTaken DESC;
```

SQL Implementation Notes

All SQL queries in this report were written and tested on a local MySQL Workbench instance. Minor syntax changes may be required for SQL Server. In a BI tool such as Power BI, similar calculations would typically be implemented using DAX measures instead of SQL. These notes apply to all queries included in this report.

2. Leave Utilization by Company & Employee

Measures:	**Total approved leave hours** Total hours of leave approved for each employee. SUM(fd.HoursTaken) **Expected working hours per employee** The total hours an employee is expected to work in a month (standard hours × working days).

	(e.StandardHoursPerDay * COUNT(*)) **Actual worked hours** Expected work hours minus approved leave hours. ExpectedWorkHours - COALESCE(ld.TotalLeaveHours, 0) **Leave utilization ratio** The percentage of an employee's working hours that are spent on leave. ROUND(COALESCE(ld.TotalLeaveHours, 0) / NULLIF(exp.ExpectedWorkHours, 0), 2)
Facts:	FactLeaveDay
Dims:	DimEmployee, DimStatus, DimDate
Explanation:	Shows how much of each employee's expected working time is spent on leave, measured by the leave utilization ratio. This helps employers track utilization at both company and employee levels, identify employees with high leave usage, and spot potential productivity risks early.

```
WITH ExpectedLeaveHours AS (
   SELECT e.EmployeeKey,
         d.Month,
         d.MonthName,
         SUM(fd.HoursTaken) AS TotalLeaveHours
   FROM FactLeaveDay fd
   JOIN DimEmployee e ON fd.EmployeeKey = e.EmployeeKey
   JOIN DimStatus s ON fd.StatusKey = s.StatusKey
   JOIN DimDate d ON fd.DateKey = d.DateKey
   WHERE s.StatusCode = 'APPROVED'
     AND YEAR(d.FullDate) = YEAR(CURDATE())
   GROUP BY e.EmployeeKey, d.Month, d.MonthName
),

ExpectedWorkHours AS (
   SELECT e.EmployeeKey,
         d.Month,
         d.MonthName,
         (e.StandardHoursPerDay * COUNT(*)) AS ExpectedWorkHours
   FROM DimEmployee e
   JOIN DimDate d ON YEAR(d.FullDate) = YEAR(CURDATE())
   WHERE d.FullDate BETWEEN e.HireDate AND COALESCE(e.TerminationDate, CURDATE())
     AND d.IsWeekend = 0
     AND d.IsHoliday = 0
   GROUP BY e.EmployeeKey, d.Month, d.MonthName, e.StandardHoursPerDay
)
```

```
SELECT e.CompanyName,
      CONCAT(e.FirstName, ' ', e.LastName) AS EmployeeName,
      exp.Month,
      exp.MonthName,
      COALESCE(ld.TotalLeaveHours, 0) AS TotalApprovedLeaveHours,
      exp.ExpectedWorkHours - COALESCE(ld.TotalLeaveHours, 0) AS TotalWorkedHours,
      ROUND(COALESCE(ld.TotalLeaveHours, 0) / NULLIF(exp.ExpectedWorkHours, 0), 2) AS
LeaveUtilizationRatio
FROM ExpectedWorkHours exp
JOIN DimEmployee e ON exp.EmployeeKey = e.EmployeeKey
LEFT JOIN ExpectedLeaveHours ld
      ON exp.EmployeeKey = ld.EmployeeKey
     AND exp.Month = ld.Month
ORDER BY e.CompanyName, EmployeeName, exp.Month;
```

3. Leave Approval Efficiency

(A) Company-Level Approval Ratios & Lead Time	
Measures:	**Approval Ratio %** The percentage of leave requests that were approved out of all requests. ApprovalRatio = ApprovedRequests / TotalRequests * 100 **Average Lead Time (Days)** Average number of days between when an employee applied for leave and when the leave actually started. AvgLeadTime = AVG(DATEDIFF(StartDate, RequestDate))
Facts:	FactLeaveRequest
Dims:	DimStatus, DimEmployee, DimDate
Explanation:	Shows how many leave requests get approved in each company and how many days it usually takes. This helps employers see if leave requests are handled quickly or if there are delays.

```
SELECT e.CompanyName,
      SUM(CASE WHEN s.StatusCode = 'APPROVED' THEN 1 ELSE 0 END) AS ApprovedRequests,
      COUNT(*) AS TotalRequests,
      ROUND(SUM(CASE WHEN s.StatusCode = 'APPROVED' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
ApprovalRatioPct,
      ROUND(AVG(DATEDIFF(ddStart.FullDate, ddReq.FullDate)), 2) AS AvgLeadTimeDays
FROM FactLeaveRequest fr
JOIN DimStatus s ON fr.StatusKey = s.StatusKey
JOIN DimEmployee e ON fr.EmployeeKey = e.EmployeeKey
JOIN DimDate ddReq ON fr.RequestDateKey = ddReq.DateKey
JOIN DimDate ddStart ON fr.StartDateKey = ddStart.DateKey
GROUP BY e.CompanyName
ORDER BY ApprovalRatioPct DESC;
```

(B) Approval Ratios & Lead Time by Leave Type	
Measures:	**Approval Ratio by Leave Type %** The percentage of approved requests out of all requests for each leave type (e.g., Annual, Sick, Unpaid). ApprovalRatioByType = ApprovedRequestsByType / TotalRequestsByType *100 **Average Lead Time by Leave Type (Days)** The average number of days between when an employee applied for leave and when the leave started, grouped by leave type. AvgLeadTimeByType = AVG(DATEDIFF(StartDate, RequestDate)) grouped by LeaveType
Facts:	FactLeaveRequest
Dims:	DimStatus, DimEmployee, DimLeaveType, DimDate
Explanation:	Shows which leave types (Annual, Sick, Unpaid, Other) are approved more often and how long approvals usually take. This helps employers see if some leave types face more delays or rejections than others.

```
SELECT e.CompanyName,
    lt.LeaveTypeCode,
    SUM(CASE WHEN s.StatusCode = 'APPROVED' THEN 1 ELSE 0 END) AS ApprovedRequests,
    COUNT(*) AS TotalRequests,
    ROUND(SUM(CASE WHEN s.StatusCode = 'APPROVED' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
ApprovalRatioPct,
    ROUND(AVG(DATEDIFF(ddStart.FullDate, ddReq.FullDate)), 2) AS AvgLeadTimeDays
FROM FactLeaveRequest fr
JOIN DimLeaveType lt ON fr.LeaveTypeKey = lt.LeaveTypeKey
JOIN DimStatus s ON fr.StatusKey = s.StatusKey
JOIN DimEmployee e ON fr.EmployeeKey = e.EmployeeKey
JOIN DimDate ddReq ON fr.RequestDateKey = ddReq.DateKey
JOIN DimDate ddStart ON fr.StartDateKey = ddStart.DateKey
GROUP BY e.CompanyName, lt.LeaveTypeCode
ORDER BY e.CompanyName, ApprovalRatioPct DESC;
```

(C) Status Breakdown (All Requests by Company)	
Measures:	**% Approved Requests** Percentage of requests that were approved out of all submitted. % Approved = Count(Approved) / TotalRequests **% Rejected Requests** Percentage of requests that were rejected.

	% Rejected = Count(Rejected) / TotalRequests **% Pending Requests** Percentage of requests still awaiting decision. % Pending = Count(Pending) / TotalRequests **% Cancelled Requests** Percentage of requests withdrawn or cancelled. % Cancelled = Count(Cancelled) / TotalRequests
Facts:	FactLeaveRequest
Dims:	DimStatus, DimEmployee
Explanation:	Shows the percentage of leave requests by status (Approved, Rejected, Pending, Cancelled) for each company. This helps employers quickly understand overall request outcomes and spot trends, like high rejection or pending rates.

```
SELECT e.CompanyName,
    ROUND(SUM(CASE WHEN s.StatusCode = 'APPROVED' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
PctApproved,
    ROUND(SUM(CASE WHEN s.StatusCode = 'REJECTED' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
PctRejected,
    ROUND(SUM(CASE WHEN s.StatusCode = 'PENDING'  THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
PctPending,
    ROUND(SUM(CASE WHEN s.StatusCode = 'CANCELLED' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
PctCancelled
FROM FactLeaveRequest fr
JOIN DimStatus s ON fr.StatusKey = s.StatusKey
JOIN DimEmployee e ON fr.EmployeeKey = e.EmployeeKey
GROUP BY e.CompanyName
ORDER BY PctApproved DESC;
```

4. Average Leave Duration per Request by Category & Company

Measures:	**Average Leave Duration** The average number of days employees take off per leave request, grouped by company and leave type (Annual, Sick, Unpaid, Other). Average Leave Duration = AVG(RequestLength) RequestLength = COUNT(DISTINCT fd.DateKey) per leave request AvgLeaveDuration = AVG(RequestLength)
Facts:	FactLeaveRequest, FactLeaveDay
Dims:	DimLeaveType, DimStatus, DimEmployee

Explanation:	Shows the average length of approved leave requests by company and leave type (Annual, Sick, Unpaid, Other). This helps employers see whether staff usually take short breaks (like 1–2 days) or longer holidays, and which types of leave are used for longer durations.

```
SELECT e.CompanyName,
      lt.LeaveTypeCode,
      AVG(RequestLength) AS AvgLeaveDuration
FROM (
   SELECT fr.LeaveRequestKey,
         fr.LeaveTypeKey,
         fr.EmployeeKey,
         COUNT(DISTINCT fd.DateKey) AS RequestLength
   FROM FactLeaveRequest fr
   JOIN FactLeaveDay fd ON fr.LeaveRequestKey = fd.LeaveRequestKey
   JOIN DimStatus s ON fr.StatusKey = s.StatusKey
   WHERE s.StatusCode = 'APPROVED'
   GROUP BY fr.LeaveRequestKey, fr.LeaveTypeKey, fr.EmployeeKey
) sub
JOIN DimLeaveType lt ON sub.LeaveTypeKey = lt.LeaveTypeKey
JOIN DimEmployee e ON sub.EmployeeKey = e.EmployeeKey
GROUP BY e.CompanyName, lt.LeaveTypeCode
ORDER BY e.CompanyName, lt.LeaveTypeCode;
```

5.Monthly Sick & Unpaid Leave Trends

Measures:	**Total Sick Leave Days** Total approved sick leave days taken by employees per month. SUM(fd.DayCount) where LeaveType = Sick) **Total Unpaid Leave Days** Total approved unpaid leave days taken by employees per month. SUM(fd.DayCount) where LeaveType = Unpaid)
Facts:	FactLeaveDay
Dims:	DimDate, DimLeaveType, DimEmployee, DimStatus
Explanation:	Shows monthly sick and unpaid leave trends by company. This helps employers spot patterns like seasonal spikes in sick leave or frequent unpaid leave, so they can plan staffing and manage productivity better.

```
SELECT e.CompanyName,
      d.Year,
      d.MonthName,
      lt.LeaveTypeCode,
      SUM(fd.DayCount) AS TotalDays
```

```
FROM FactLeaveDay fd
JOIN DimDate d ON fd.DateKey = d.DateKey
JOIN DimLeaveType lt ON fd.LeaveTypeKey = lt.LeaveTypeKey
JOIN DimStatus s ON fd.StatusKey = s.StatusKey
JOIN DimEmployee e ON fd.EmployeeKey = e.EmployeeKey
WHERE s.StatusCode = 'APPROVED'
  AND lt.LeaveTypeCode IN ('SICK','UNPAID')
GROUP BY e.CompanyName, d.Year, d.MonthName, lt.LeaveTypeCode
ORDER BY e.CompanyName, d.Year, d.MonthName, lt.LeaveTypeCode;
```

6. Leave Application Lead Time (Last-Minute vs Planned)

Measures:	**Average Lead Time (Days)** Average gap between when employees apply and when their leave starts. Average lead time = AVG(DATEDIFF(StartDate, RequestDate)) **Last-Minute Requests** Number of leave requests made 1 day or less before leave starts. **Planned-in-Advance Requests** Number of leave requests submitted more than 7 days ahead. **Total Requests** Total leave requests submitted by each employee. Total requests = COUNT(*)
Facts:	FactLeaveRequest
Dims:	DimEmployee, DimDate
Explanation:	Shows how far in advance employees apply for leave. This highlights whether leave requests are mostly last-minute or well-planned, helping employers improve scheduling, manage workloads, and reduce disruptions caused by unexpected absences.

```
SELECT e.CompanyName,
    e.EmployeeID,
    e.FirstName,
    e.LastName,
    AVG(DATEDIFF(ddStart.FullDate, ddReq.FullDate)) AS AvgLeadTimeDays,
    SUM(CASE WHEN DATEDIFF(ddStart.FullDate, ddReq.FullDate) <= 1 THEN 1 ELSE 0 END) AS
LastMinuteRequests,
    SUM(CASE WHEN DATEDIFF(ddStart.FullDate, ddReq.FullDate) > 7 THEN 1 ELSE 0 END) AS
PlannedInAdvanceRequests,
    COUNT(*) AS TotalRequests
FROM FactLeaveRequest fr
JOIN DimEmployee e ON fr.EmployeeKey = e.EmployeeKey
JOIN DimDate ddReq ON fr.RequestDateKey = ddReq.DateKey
```

```
JOIN DimDate ddStart ON fr.StartDateKey = ddStart.DateKey
GROUP BY e.CompanyName, e.EmployeeID, e.FirstName, e.LastName
ORDER BY e.CompanyName, AvgLeadTimeDays ASC;
```

7. Current & Upcoming Coverage (Next 30 days)

Measures:	**Employees on Leave (Next 30 Days)** List of employees who will be absent in the next 30 days, along with their leave dates. Employees on leave per day = COUNT(DISTINCT EmployeeKey) Employee details = (ID, Name, Company)
Facts:	FactLeaveDay
Dims:	DimEmployee, DimDate, DimStatus
Explanation:	Shows which employees are on approved leave in the next 30 days, along with their company and leave dates. This helps employers plan ahead, manage staffing levels, and ensure enough coverage when multiple employees are away at the same time.

```
SELECT e.CompanyName,
      d.FullDate,
      e.EmployeeID,
      e.FirstName,
      e.LastName
FROM FactLeaveDay fd
JOIN DimEmployee e ON fd.EmployeeKey = e.EmployeeKey
JOIN DimDate d ON fd.DateKey = d.DateKey
JOIN DimStatus s ON fd.StatusKey = s.StatusKey
WHERE s.StatusCode = 'APPROVED'
  AND d.FullDate BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 30 DAY)
ORDER BY e.CompanyName, d.FullDate, e.LastName, e.FirstName;
```

8. Monthly Leave Seasonality

Measures:	**Total Leave Requests** Number of leave applications submitted each month. Total leave requests = COUNT(DISTINCT LeaveRequestKey) **Total Leave Days** Total days of leave (across all requests) in each month. Total leave days = SUM(fd.DayCount)

Facts:	FactLeaveRequest, FactLeaveDay
Dims:	DimEmployee, DimDate, DimStatus
Explanation:	Shows leave activity by month for each company, including all applied leaves (approved, rejected, pending, etc.). This helps employers identify peak months when leave requests are high and quieter periods with fewer requests, making it easier to plan workloads and resource allocation across the year.

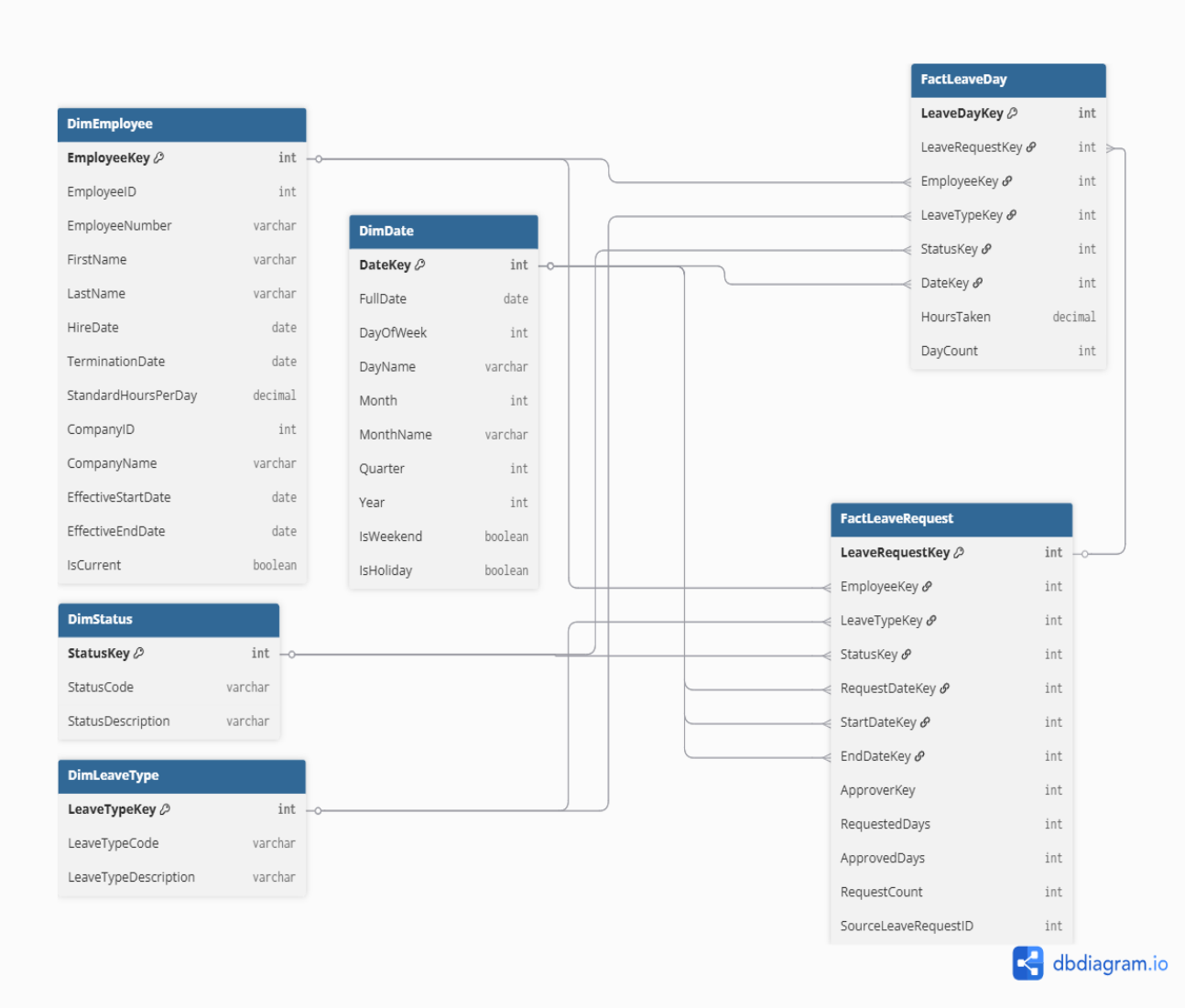
```
SELECT e.CompanyName,
      d.Month,
      d.MonthName,
      COUNT(DISTINCT fr.LeaveRequestKey) AS TotalLeaveRequests,
      SUM(fd.DayCount) AS TotalLeaveDays
FROM FactLeaveRequest fr
JOIN FactLeaveDay fd ON fr.LeaveRequestKey = fd.LeaveRequestKey
JOIN DimEmployee e ON fr.EmployeeKey = e.EmployeeKey
JOIN DimDate d ON fd.DateKey = d.DateKey
JOIN DimStatus s ON fr.StatusKey = s.StatusKey
GROUP BY e.CompanyName, d.Month, d.MonthName
ORDER BY e.CompanyName, TotalLeaveDays DESC;
```

9. Leave Before Resignation/Termination

Measures:	**Leave Days in Last 90 Days** Total approved leave days an employee took in the 90 days before their termination/resignation date. Leave days in last 90 days before termination = SUM(fd.DayCount)
Facts:	FactLeaveDay
Dims:	DimEmployee, DimDate, DimStatus
Explanation:	Shows how many leave days employees take in the 90 days before leaving the company. This helps employers notice if someone takes a lot of leave just before resigning or being terminated.

```
SELECT e.EmployeeID, e.FirstName, e.LastName, e.CompanyName,
      e.TerminationDate,
      SUM(fd.DayCount) AS LeaveDaysLast3Months
FROM DimEmployee e
JOIN FactLeaveDay fd ON e.EmployeeKey = fd.EmployeeKey
JOIN DimDate d ON fd.DateKey = d.DateKey
JOIN DimStatus s ON fd.StatusKey = s.StatusKey
WHERE e.TerminationDate IS NOT NULL
  AND d.FullDate BETWEEN DATE_SUB(e.TerminationDate, INTERVAL 90 DAY) AND e.TerminationDate
  AND s.StatusCode = 'APPROVED'
GROUP BY e.EmployeeID, e.FirstName, e.LastName, e.CompanyName, e.TerminationDate
ORDER BY e.TerminationDate DESC;
```

Part B — Dimensional model



Rationale for Insights

The business mainly needs two types of analysis:

1. **Leave request level** – how employees apply for leave, the types of leave they take, the approval or rejection of requests, and the efficiency of the overall process.
2. **Day level** – how approved leave translates into actual absences on specific days, which directly impacts coverage and workforce planning.

Because of these two key events or transactions, I designed two fact tables:

- **FactLeaveDay**: for day-level analysis.
- **FactLeaveRequest**: for request-level analysis.

Having two fact tables keeps the right level of granularity and avoids forcing a single table to handle both jobs.

I merged company information into DimEmployee to denormalize and avoid creating a separate dimension, since star schemas are meant to store denormalized data. This way, when analyzing employee leave, company details can be filtered directly from DimEmployee. This design also helps BI tools retrieve data faster and more easily from a single table.

- **DimEmployee** is modeled as **SCD2**, tracking effective_start_date, effective_end_date, and is_current status, so that historical changes can be captured.
- **DimDate** is a full calendar table with year, quarter, and month attributes, which allows analysis of trends, lead time between request and approval, absenteeism windows, and more.
- **DimLeaveType** is modeled as **SCD1**, since leave types usually remain stable and do not require historical tracking.
- **DimStatus** is also **SCD1**, because approval status codes are business process outcomes rather than historical attributes.

Note: A star schema is simpler for end users because it requires fewer dimensions and fewer joins.

Part C — Build the fact (SQL)

FactLeaveRequest

```
INSERT INTO FactLeaveRequest
(EmployeeKey, LeaveTypeKey, StatusKey, RequestDateKey, StartDateKey, EndDateKey,
 ApproverKey, RequestedDays, ApprovedDays, RequestCount, SourceLeaveRequestID)
SELECT de.EmployeeKey,
      dlt.LeaveTypeKey,
      ds.StatusKey,
      ddReq.DateKey,
      ddStart.DateKey,
      ddEnd.DateKey,
      NULL,
      DATEDIFF(lr.EndDate, lr.StartDate) + 1,
      CASE WHEN lr.StatusCode='APPROVED'
          THEN (DATEDIFF(lr.EndDate, lr.StartDate) + 1) ELSE 0 END,
      1,
      lr.LeaveRequestID
FROM LeaveRequest lr
JOIN DimEmployee de ON lr.EmployeeID = de.EmployeeID
JOIN DimLeaveType dlt ON lr.LeaveTypeCode = dlt.LeaveTypeCode
JOIN DimStatus ds ON lr.StatusCode = ds.StatusCode
JOIN DimDate ddReq ON ddReq.FullDate = lr.RequestDate
JOIN DimDate ddStart ON ddStart.FullDate = lr.StartDate
JOIN DimDate ddEnd ON ddEnd.FullDate = lr.EndDate;
```

FactLeaveDay

```
INSERT INTO FactLeaveDay
(LeaveRequestKey, EmployeeKey, LeaveTypeKey, StatusKey, DateKey, HoursTaken, DayCount)
SELECT fr.LeaveRequestKey,
      fr.EmployeeKey,
      fr.LeaveTypeKey,
      fr.StatusKey,
      dd.DateKey,
      ld.Hours,
      1
FROM LeaveDay ld
JOIN LeaveRequest lr ON ld.LeaveRequestID = lr.LeaveRequestID
JOIN FactLeaveRequest fr ON lr.LeaveRequestID = fr.SourceLeaveRequestID
JOIN DimDate dd ON dd.FullDate = ld.LeaveDate;
```

Note: *The SQL scripts were written and tested in MySQL Workbench 8.0 using a local instance. Since the preferred SQL dialect is SQL Server, there may be slight syntax differences.*